

9주차

CHAPTER7 합성곱 신경망 (CNN)

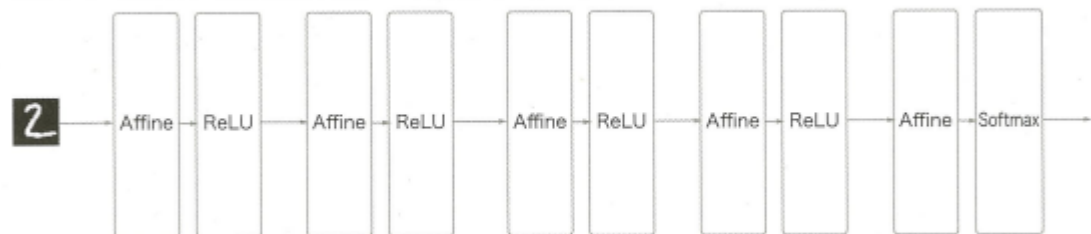
7.1. 전체 구조

CNN = 신경망 + 합성곱 계층 + 풀링 계층

지금까지는 인접 계층의 모든 뉴런과 연결된 " 완전 연결 " 을 보았다

이렇게 완전히 연결된 계층을 Affine 계층이라는 이름으로 구현하였는데

그림 7-1 완전연결 계층(Affine 계층)으로 이뤄진 네트워크의 예

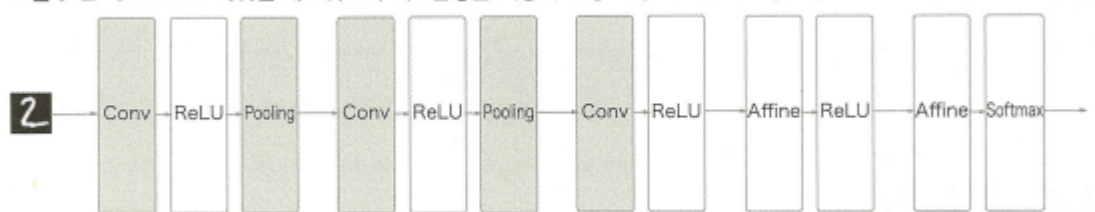


Affine 계층을 사용한 모습은 다음과 같다

마지막 계층에서 **softmax**가 나오는데 여기서 최종 결과인 **확률을 출력**한다.

그렇다면 CNN은?

그림 7-2 CNN으로 이뤄진 네트워크의 예 : 합성곱 계층과 풀링 계층이 새로 추가(회색)



다음과 같이 Affine 대신 합성곱 계층 (Conv)과 풀링 계층(Pooling)이 추가된다

7.2. 합성곱 계층

CNN에서 사용하는 계층 구조 살펴보기

7.2.1. 완전연결 계층의 문제점

Affine 계층의 문제점 $\Rightarrow$ data의 형상이 무시된다

ex) data가 3차원 구조일 때 (이미지일 경우 높이 x 너비 x 채널(색상))인데

3차원 $\rightarrow$ 1차원 데이터로 평탄화 해줘야 함

MNIST data에서는 1채널 x 28픽셀 x 28픽셀 == 784개의 data

이렇게 1차원 data로 만든 784개의 data를 첫 Affine 계층에 입력했다.

이미지는 3차원 형상이기 때문에 공간적 정보가 담겨있는데 완전 연결 계층은 이를 모두 무시한다

하지만, 합성곱 계층 CNN은 형상을 유지한다.

3차원 data 입력 $\rightarrow$ 3차원 data로 전달 $\rightarrow$...

CNN에서 합성곱 계층의 입출력 data를 특징 맵(feature map)이라고 한다

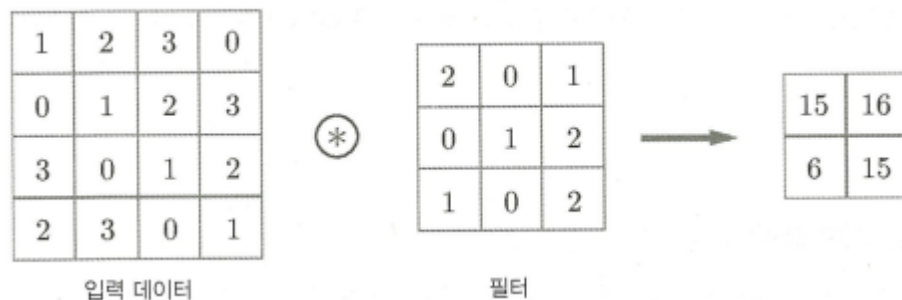
$\rightarrow$ 합성곱 계층의 입력 data $\Rightarrow$ 입력 특징 맵

$\rightarrow$ 합성곱 계층의 출력 data $\Rightarrow$ 출력 특징 맵

7.2.2. 합성곱 연산

합성곱 연산은 이미지 처리에서 말하는 "필터 연산"이다.

그림 7-3 합성곱 연산의 예 : 합성곱 연산을 $\otimes$ 기호로 표기



다음과 같이 입력 data에 필터를 적용한다

data와 필터의 형상을 (높이 height, 너비 width)로 표기

위의 그림에서는 입력(4,4) 필터(3,3) 출력(2,2)

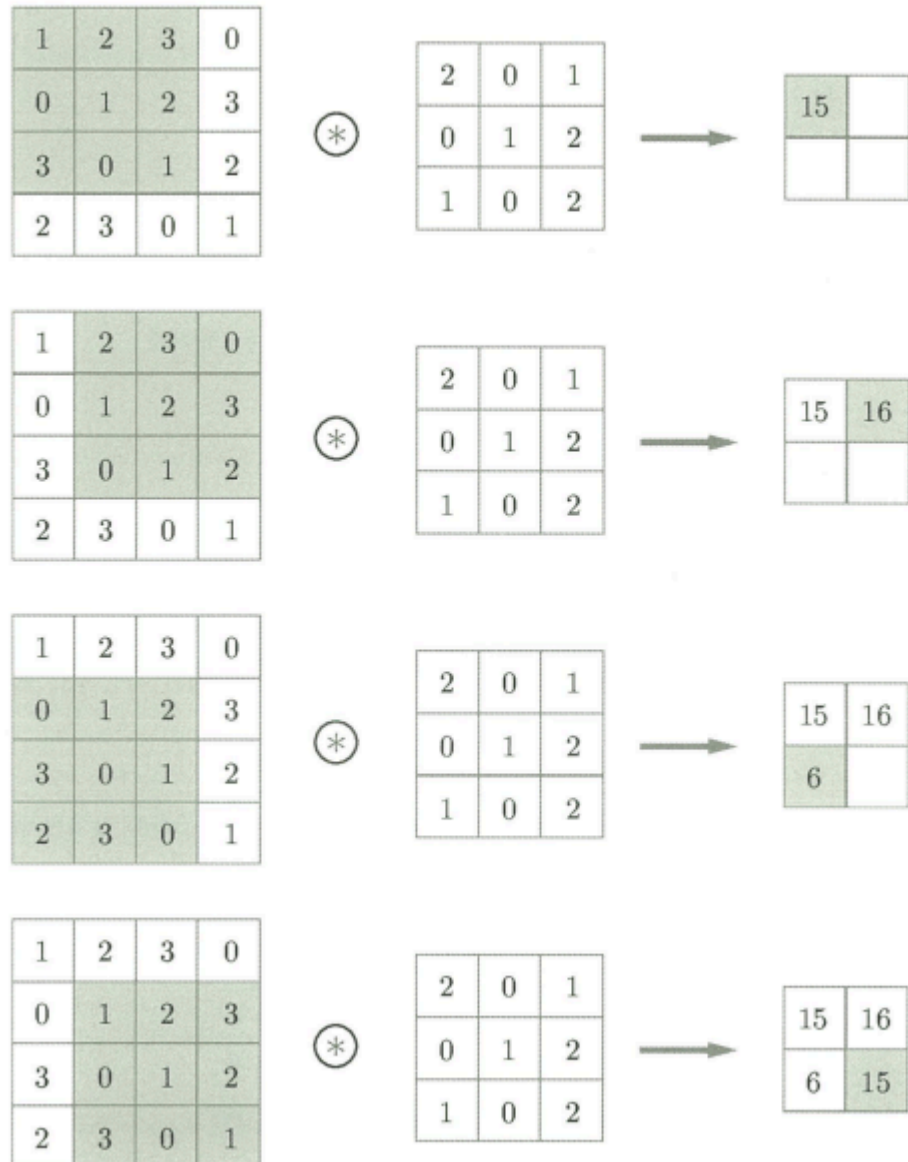
합성곱 연산은 필터의 윈도우(window)를 일정 간격 이동하며 입력 data에 적용한다.

window = 필터 (위의 그림에서 3x3)

아래의 연산을 모든 부분에서 수행하면 합성곱 연산의 출력이 완성

입력과 필터에서 대응하는 원소끼리 곱 → 총 합을 구한다(= FMA) → 결과를 출력 장소에 저장

그림 7-4 합성곱 연산의 계산 순서

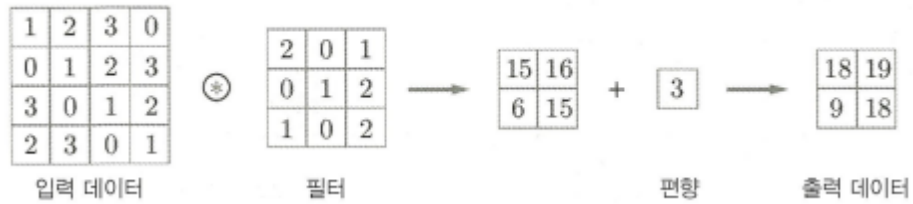


완전연결 신경망에는 가중치 매개변수 & 편향이 있다.

CNN에서는 필터의 매개변수가 그동안의 "가중치"에 해당한다.

또한 CNN에서도 "편향"이 존재한다.

그림 7-5 합성곱 연산의 편향 : 필터를 적용한 원소에 고정값(편향)을 더한다.



편향

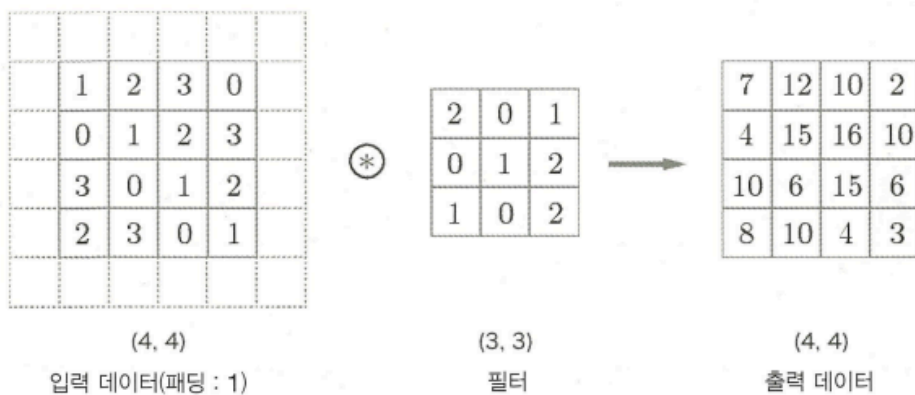
1. 필터를 적용한 후 데이터에 더해지며
2. 항상 하나 (1x1)만 존재
3. 그 하나의 값을 필터를 적용한 모든 원소에 더함

7.2.3. 패딩

합성곱 연산 수행 전 입력 data 주변을 특정 값(ex] 0)으로 채우는 기법

따라서 기본 크기가 (n,n)일경우 폭 1 패딩 적용시 (n+2, n+2)?

그림 7-6 합성곱 연산의 패딩 처리 : 입력 데이터 주위에 0을 채운다(패딩은 점선으로 표시했으며 그 안의 값 '0'은 생략했다).



패딩을 사용하는 이유 :

보통 출력 크기를 조정하기 위해 사용

(4,4)인데 (3,3)필터를 적용하면, (2,2) 출력이 나옴 ⇒ 입력보다 2만큼 줄어서 합성곱 연산을 수차례 되풀이하는 심층 신경망에서는 문제가 될 수 있음

⇒ 어느순간 출력 크기가 1이 되어버려서 더이상 합성곱을 적용할 수 없음

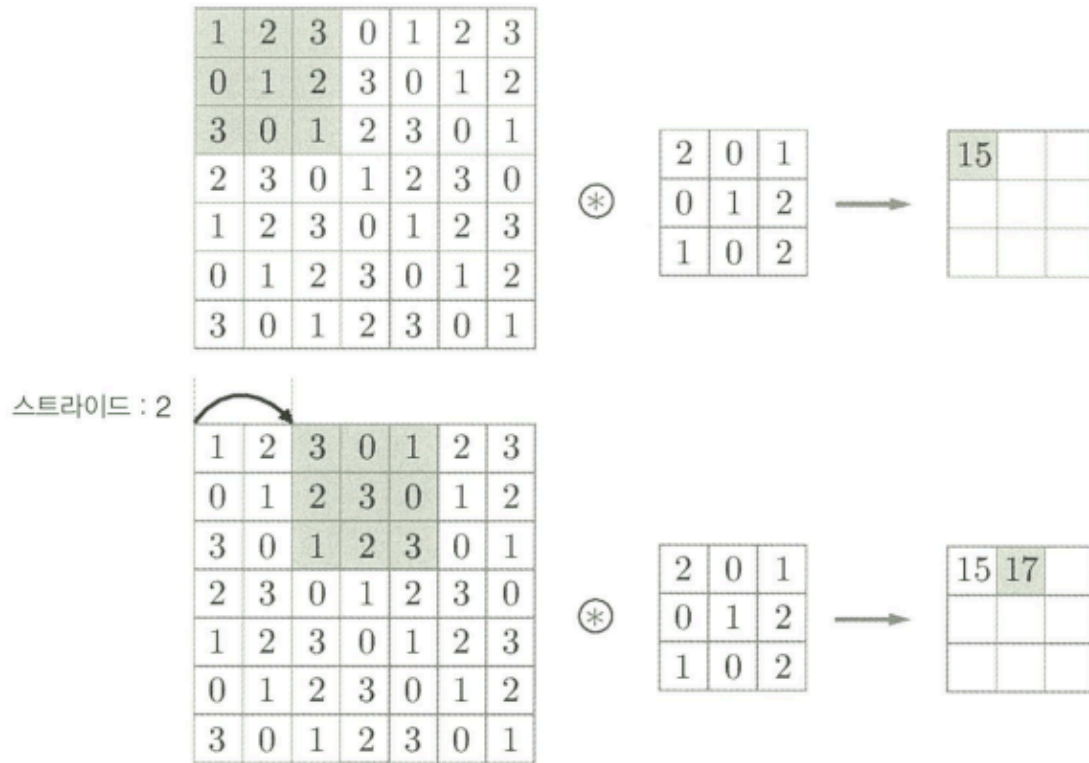
7.2.4 스트라이드

: 필터적용 간격

위의 예까지는 스트라이드가 1이었는데, 조정이 가능함

2로 하게되면 적용하는 윈도우가 2칸씩 이동함

그림 7-7 스트라이드가 2인 합성곱 연산



⇒ 스트라이드를 키우면 → 출력 크기가 작아짐

(7,7) → 스트라이드 2 → (3,3)

(n,n) → 스트라이드 x → (n-x*2, n-x*2)?

$$OH = \frac{H + 2P - FH}{S} + 1$$

$$OW = \frac{W + 2P - FW}{S} + 1$$

예 1 : [그림 7-6]의 예

입력 : (4, 4), 패딩 : 1, 스트라이드 : 1, 필터 : (3, 3)

$$OH = \frac{4 + 2 \cdot 1 - 3}{1} + 1 = 4$$

$$OW = \frac{4 + 2 \cdot 1 - 3}{1} + 1 = 4$$

예 2 : [그림 7-7]의 예

입력 : (7, 7), 패딩 : 0, 스트라이드 : 2, 필터 : (3, 3)

$$OH = \frac{7 + 2 \cdot 0 - 3}{2} + 1 = 3$$

$$OW = \frac{7 + 2 \cdot 0 - 3}{2} + 1 = 3$$

예 3

입력 : (28, 31), 패딩 : 2, 스트라이드 : 3, 필터 : (5, 5)

$$OH = \frac{28 + 2 \cdot 2 - 5}{3} + 1 = 10$$

$$OW = \frac{31 + 2 \cdot 2 - 5}{3} + 1 = 11$$

7.2.5. 3차원 데이터의 합성곱 연산

채널쪽 차원의 특징맵이 여러개라면,

입력 데이터와 필터의 합성곱 연산을 채널마다 수행

→ 결과끼리 합

→ 하나의 출력

그림 7-8 3차원 데이터 합성곱 연산의 예

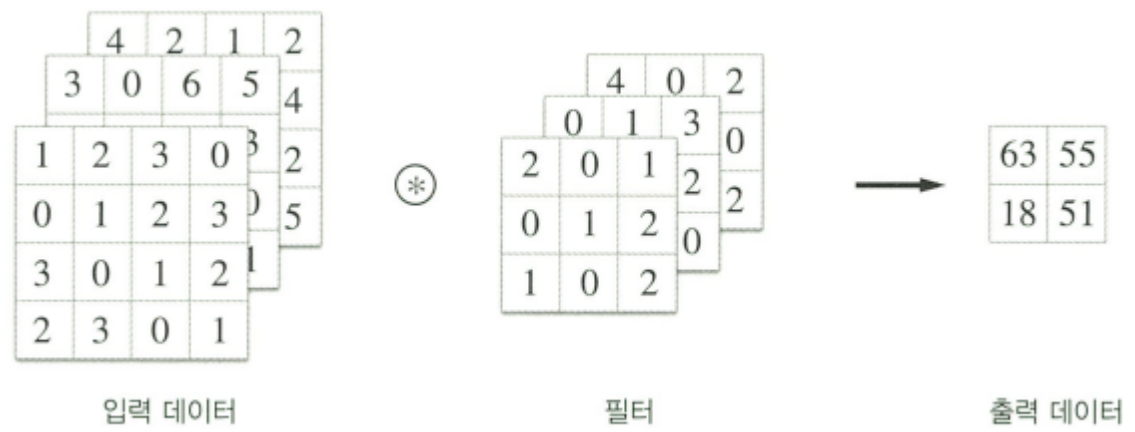
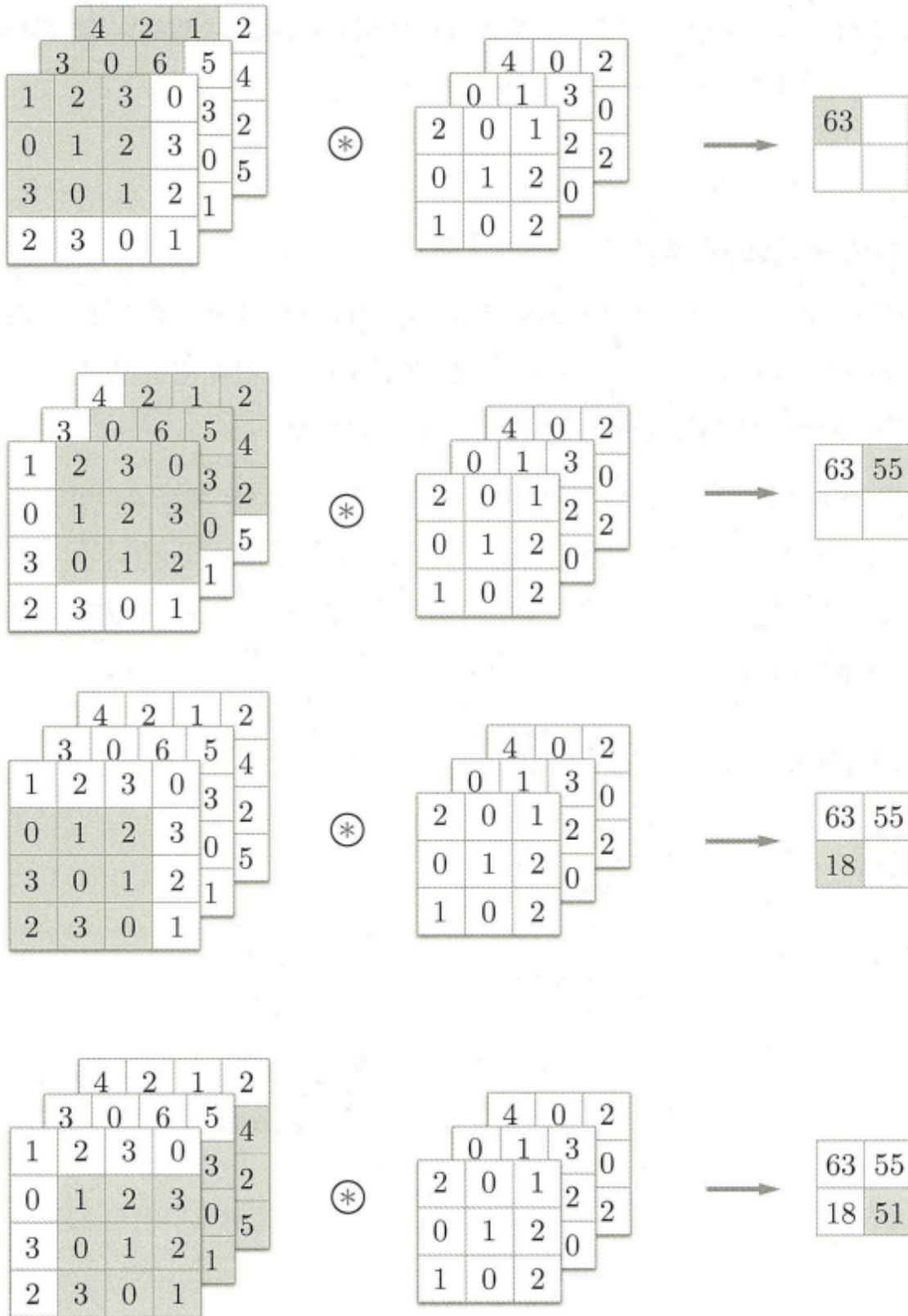


그림 7-9 3차원 데이터 합성곱 연산의 계산 순서



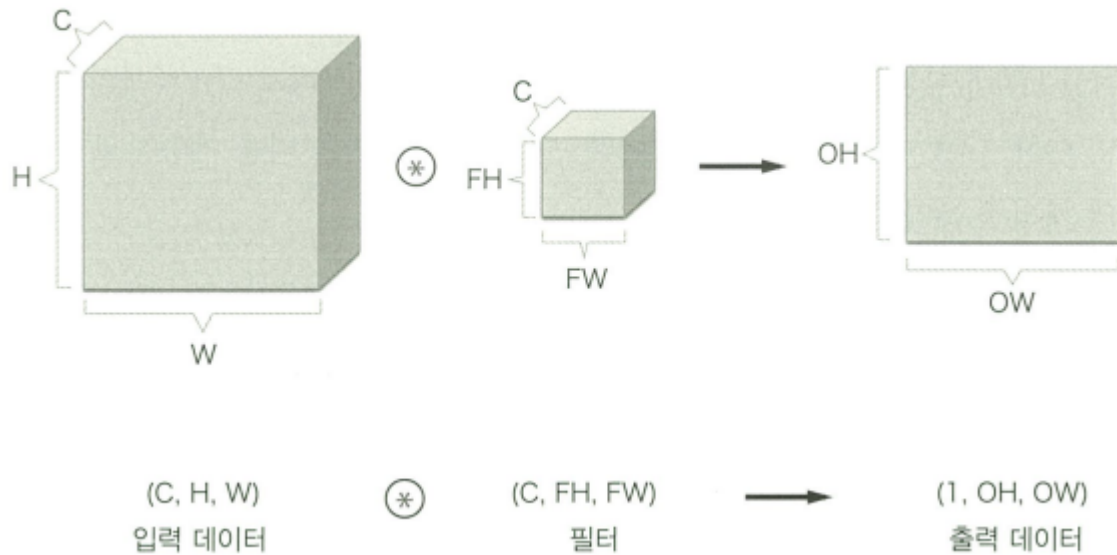
참고로 위의 그림과 같이 입력 data의 채널 수와 필터로 적용할 채널의 갯수가 동일해야함

7.2.6. 블록으로 생각해보기

직육면체 블록이라고 생각해서 계산하기

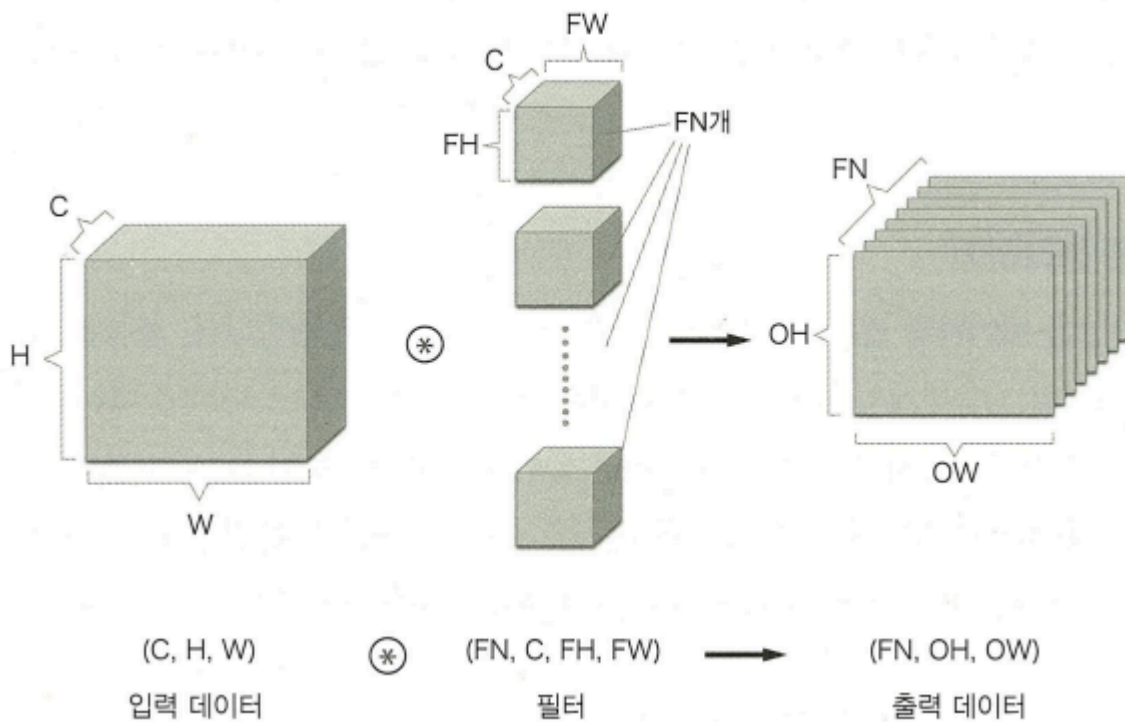
7.2.5. 를 보충 설명

그림 7-10 합성곱 연산을 직육면체 블록으로 생각한다. 블록의 형상에 주의할 것!



필터의 갯수에 따라 출력 갯수가 결정된다.

그림 7-11 여러 필터를 사용한 합성곱 연산의 예



필터를 FN개 설정한 data는 출력 데이터 층이 FN개가 나옴

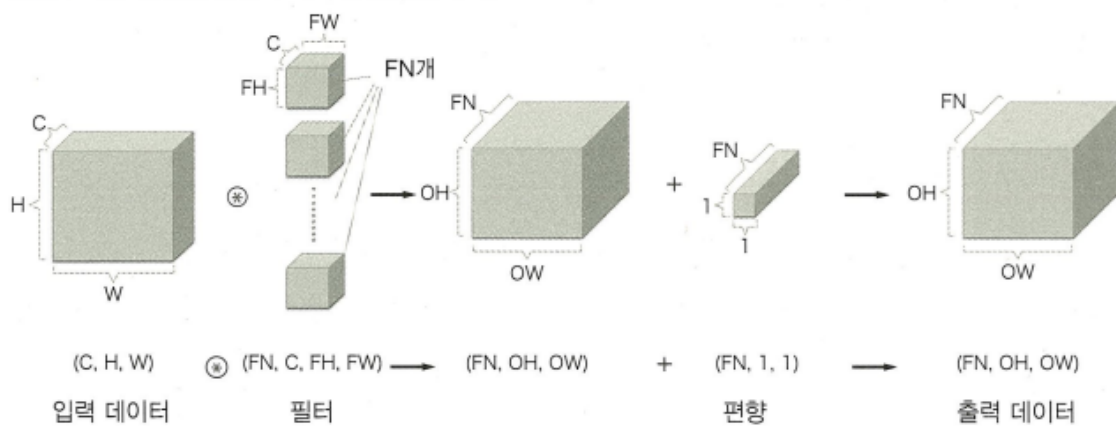
→ 필터의 수도 고려해야함

필터의 가중치 데이터는 필터의 수도 고려하기위해 4차원 data가 된다.

(출력 채널 갯수, 입력 채널 갯수, 높이, 너비)

여기서 편향 추가하는 방법

그림 7-12 합성곱 연산의 처리 흐름(편향 추가)



7.2.7. 배치 처리

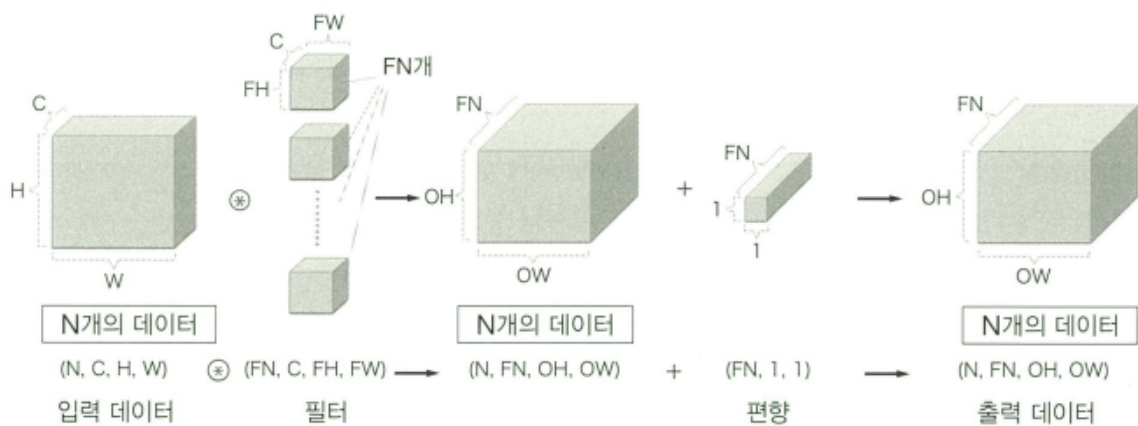
완전연결 신경망 구현시 처리 효율을 높이며 미니배치 방식의 학습도 지원하도록 한 처리

합성곱 연산에서도 사용할 수 있음

총 4차원 데이터로 저장(데이터 수, 채널 수, 높이, 너비) 순서

n개의 data에서

그림 7-13 합성곱 연산의 처리 흐름(배치 처리)



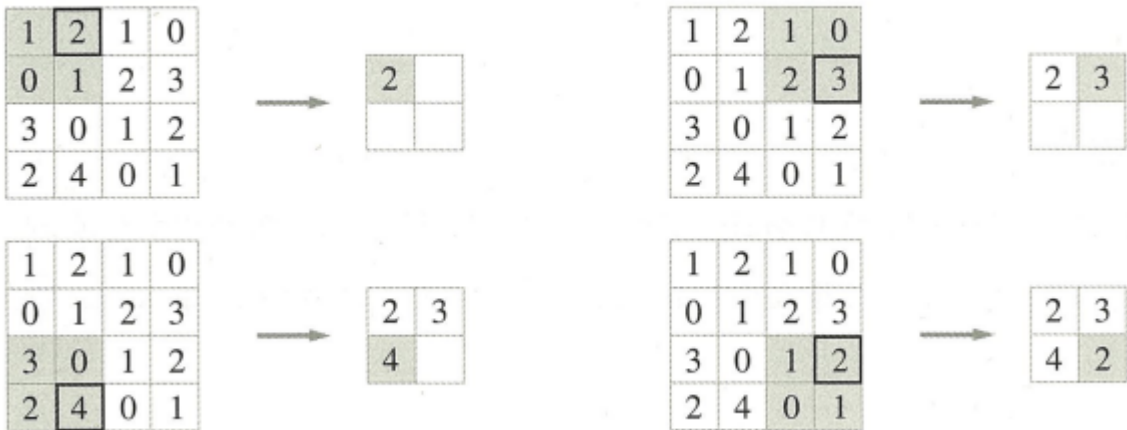
참고로 4차원 data 하나 흐름때마다 N개에 대해 합성곱 연산이 이루어져서 N회분의 처리를 한 번에(?) 수행!

7.3. 풀링 계층

가로와 세로 방향의 공간 줄이기 $\Rightarrow$ 하나로 집약하는 것

최대, 최소, 평균 풀링 등이 있음

그림 7-14 최대 풀링의 처리 순서



다음 그림은 2x2 최대 풀링을 스트라이드 2를 적용하여 처리한 것

보통 🐡

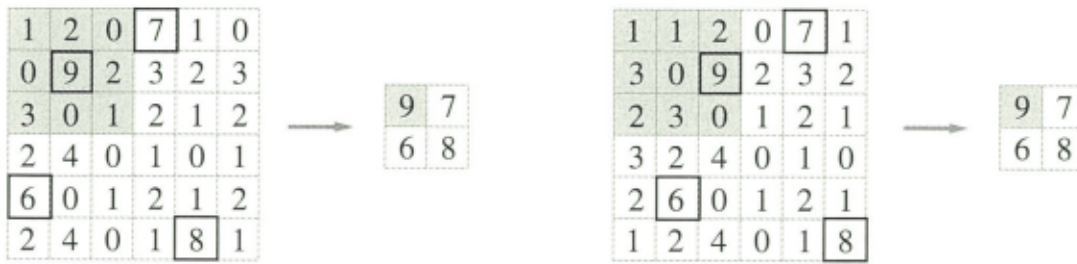
풀링의 윈도우 크기 == 스트라이드 크기

ex) 윈도우가 3x3이면 스트라이드도 3으로 설정

7.3.1. 풀링 계층의 특징

1. 학습할 매개변수가 없음
 - a. 합성곱 계층과 달리 학습해야 할 매개변수가 없음
 - b. 대다 영역에서 그냥 최댓값, 평균을 취하는 **명확한 처리**
2. 채널 수 변하지 않음
 - a. 입력 data의 채널 수 그대로를 출력 data로 내보냄
 - b. 채널마다 독립적으로 계산되기 때문
3. 입력의 변화에 적은 영향을 받음 ⇒ 강건하다 💪
 - a. 입력 data가 조금 변하더라도 풀링 결과가 잘 변하지 않는다.
 - b. 작은 입력 data의 차이는 풀링이 흡수하여 사라지게 만들기 때문

그림 7-16 입력 데이터가 가로로 1원소만큼 어긋나도 출력은 같다(데이터에 따라서는 다를 수도 있다).



7.4. 합성곱/풀링 계층 구현하기

파이썬으로 구현하기

```

import numpy as np
import random

x = np.random.rand(10,1,28,28)
#데이터, 채널, 높이, 너비

[3] x[0].shape
x[1].shape

(1, 28, 28)

[7] x[0,0]

1.30960359e-01, 2.81162532e-01, 5.38791965e-01, 8.60168563e-01,
9.21139526e-02, 1.82197942e-01, 3.77378037e-01, 6.27367639e-01],
[4.10835201e-01, 1.03951734e-01, 7.07768816e-01, 9.59981924e-01,
2.69896355e-01, 6.52827607e-04, 9.72898569e-01, 8.38627602e-01,
4.86262111e-01, 8.51841083e-01, 2.28385688e-01, 2.61960948e-01,
3.89928819e-01, 5.98203706e-01, 3.47373048e-02, 7.34395103e-01,
1.05044665e-02, 9.35718154e-01, 6.42090129e-01, 8.92528147e-01,
3.73898934e-01, 3.96779268e-01, 1.38218792e-01, 7.40348840e-01,
3.18935229e-01, 3.80759579e-01, 7.00033673e-01, 9.15261344e-01],
[6.44257144e-01, 2.89252519e-01, 4.97460637e-01, 6.41174848e-01,
3.93668800e-01, 5.76846310e-01, 2.80239836e-01, 6.44073859e-02,
5.36852545e-01, 5.50378289e-01, 1.32751516e-01, 7.46886893e-01,

```

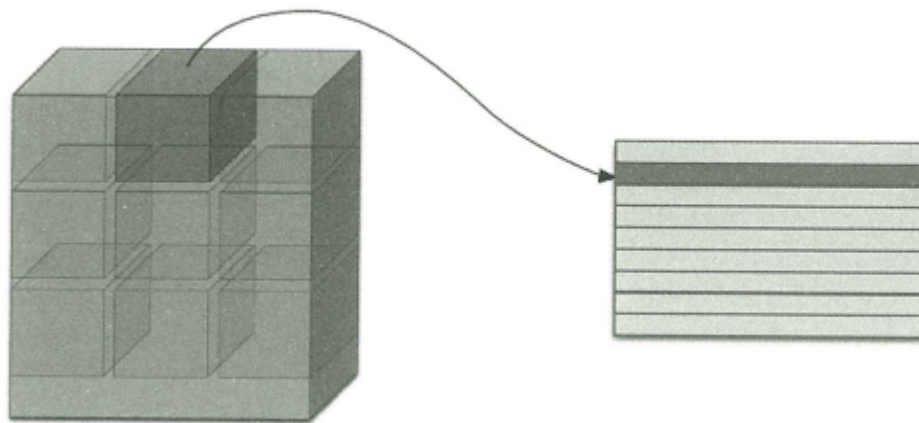
여기서 합성곱 연산을 구현 → but for문을 겹겹이 써야함 → im2col이라는 함수를 사용 → 펼치기 함수

3차원(정확히는 데이터 수까지 4차원) — im2col → 2차원

데이터를 필터링하기 좋게 전개하고 입력 데이터에서 필터를 적용하는 영역을 한 줄로 늘어둠

→ 이 전개를 필터를 적용하는 모든 영역에서 수행

그림 7-18 필터 적용 영역을 앞에서부터 순서대로 1줄로 펼친다.



단점은 메모리를 더 많이 소비한다는 점

but 컴퓨터는 큰 행렬을 묶어 계산하는데에 탁월함

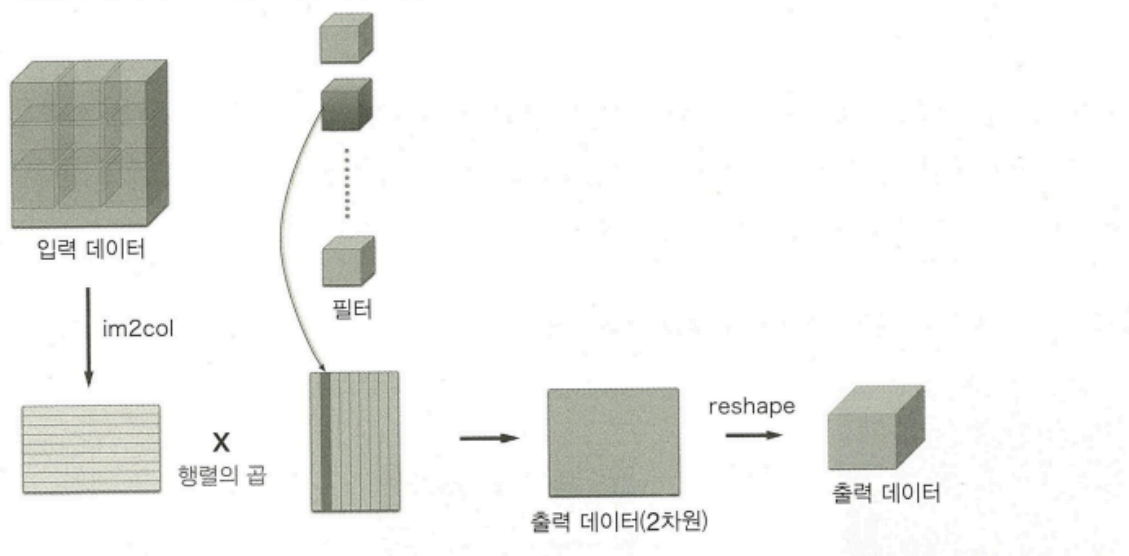
ex 행렬 계산 라이브러리는 행렬 계산에 매우 최적화 되어 있어 큰 행렬의 곱셈을 빠르게 계산 가능

⇒ 행렬 계산으로 만들면 선형대수 라이브러리를 활용하여 높은 효율을 낼 수 있음

im2col 전개 이후 합성곱 계층의 가중치(필터)를 1열로 전개 ⇒ 두 행렬의 곱을 계산

Affine 계층에서 한 것과 유사

그림 7-19 합성곱 연산의 필터 처리 상세 과정 : 필터를 세로로 1열로 전개하고, im2col이 전개한 데이터와 행렬 곱을 계산합니다. 마지막으로 출력 데이터를 변형(reshape)합니다.



7.4.3. 합성곱 계층 구현하기

```
im2col(input_data, filter_h, filter_w, stride=1, pad=0)
```

- input_data – (데이터 수, 채널 수, 높이, 너비)의 4차원 배열로 이뤄진 입력 데이터
- filter_h – 필터의 높이
- filter_w – 필터의 너비
- stride – 스트라이드
- pad – 패딩

다음과 같이 만들어진 im2col을 사용할 예정

```
import sys, os
sys.path.append(os.pardir)
from common.util import im2col

x1 = np.random.rand(1, 3, 7, 7) # (데이터 수, 채널 수, 높이, 너비)
col1 = im2col(x1, 5, 5, stride=1, pad=0)
print(col1.shape) # (9, 75)

x2 = np.random.rand(10, 3, 7, 7) # 데이터 10개
col2 = im2col(x2, 5, 5, stride=1, pad=0)
print(col2.shape) # (90, 75)
```

x1은 배치1 채널3 높이7 너비7

x2는 배치10, 채널3 높이7 너비7

⇒ 배치 크기만 차이 있음

○합성곱 구현

```

class Convolution:
    def __init__(self, W, b, stride=1, pad=0):
        self.W = W
        self.b = b
        self.stride = stride
        self.pad = pad

    def forward(self, x):
        FN, C, FH, FW = self.W.shape
        N, C, H, W = x.shape
        out_h = int(1 + (H + 2*self.pad - FH) / self.stride)
        out_w = int(1 + (W + 2*self.pad - FW) / self.stride)

        col = im2col(x, FH, FW, self.stride, self.pad)
        col_W = self.W.reshape(FN, -1).T # 필터 전개
        out = np.dot(col, col_W) + self.b

        out = out.reshape(N, out_h, out_w, -1).transpose(0, 3, 1, 2)

        return out

```

합성곱 계층은 필터, 편향, 스트라이드, 패딩을 인수로 받고 초기화함
 이때 필터는 필터 갯수, 채널, 높이, 너비를 말하는 4차원 형상

bold체 되어있는 부분이 핵심

입력 데이터를 **im2col**로 전개하고

필터도 **reshape**를 활용하여 2차원 배열로 전개

전개한 두 행렬의 곱 구하기

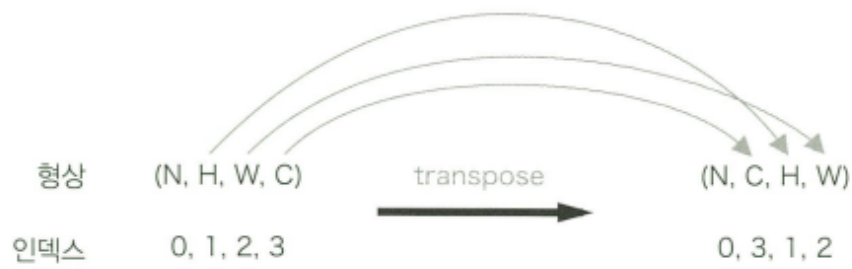
⇒ 각 필터의 블록이 1줄로 펼쳐세워짐

forward 구현 마지막에선 출력 **data**를 원하는 형상으로 바꿔준다

⇒ **transpose** 함수 사용

⇒ 다차원 배열의 축 순서를 변경함

그림 7-20 넘파이의 transpose 함수로 축 순서 변경하기 : 인덱스(번호)로 축의 순서를 변경한다.

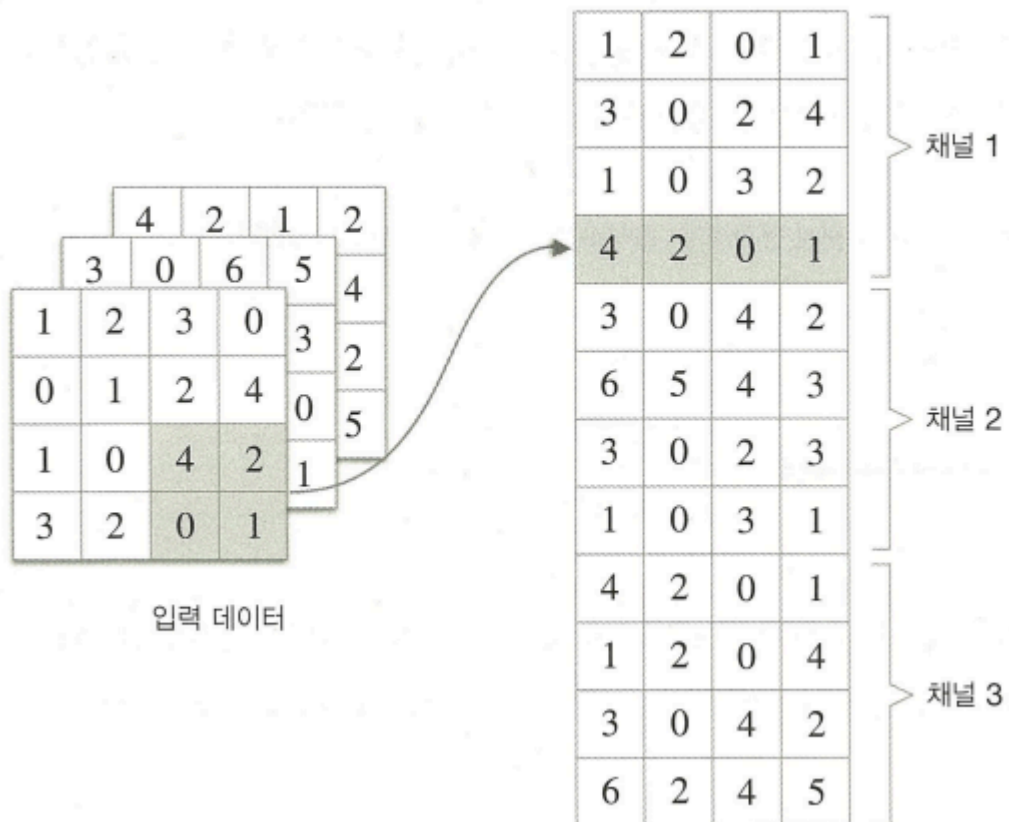


7.4.4. 풀링 계층 구현하기

합성곱 계층과 마찬가지로 im2col을 사용하여 입력 데이터 전개

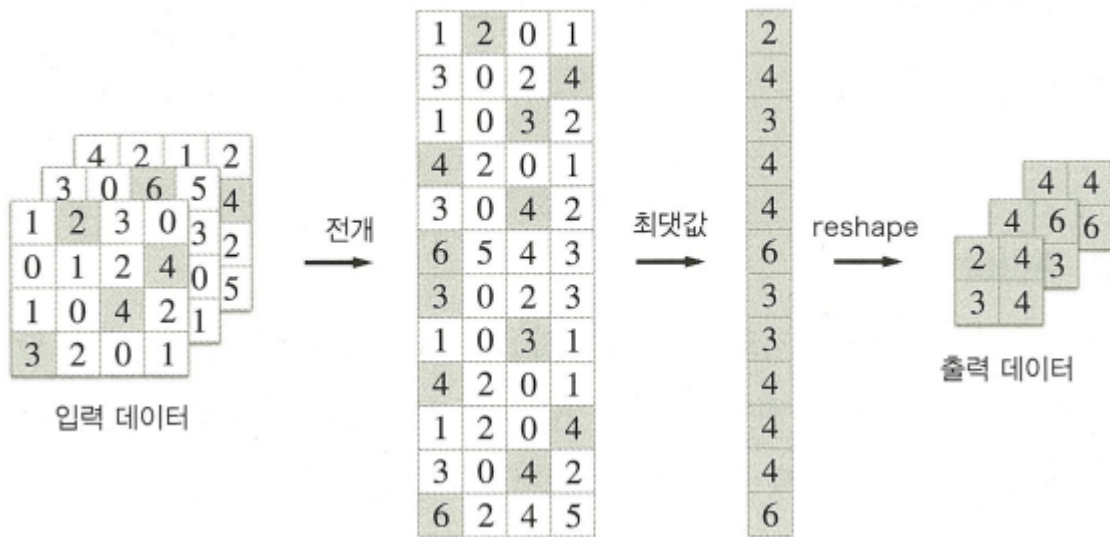
풀링은 채널이 독립적이라는 것만 다르고 나머지는 동일

그림 7-21 입력 데이터에 풀링 적용 영역을 전개 (2×2 풀링의 예)



묶음별로 한 줄씩 영역 전개

그림 7-22 풀링 계층 구현의 흐름 : 풀링 적용 영역에서 가장 큰 원소는 회색으로 표시



전개한 행렬에서 행별 최댓값(max 풀링 적용시) 착안 → 최종본은 원하는 형상으로, 적절하게 성형하기

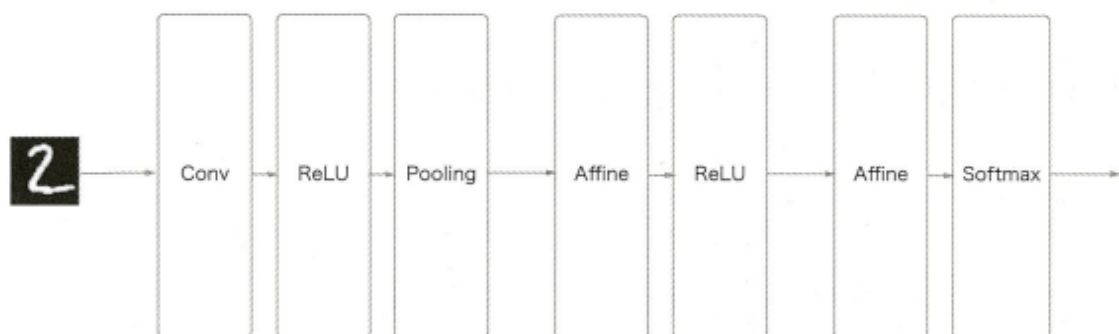
따라서 풀링 계층 구현의 3단계는

1. 입력 데이터 전개
2. 행별 최댓값 구하기
3. 적절한 모양으로 성형

7.5. CNN 구현하기

손글씨 숫자를 인식하는 CNN 조립하기

그림 7-23 단순한 CNN의 네트워크 구성



오류 확인

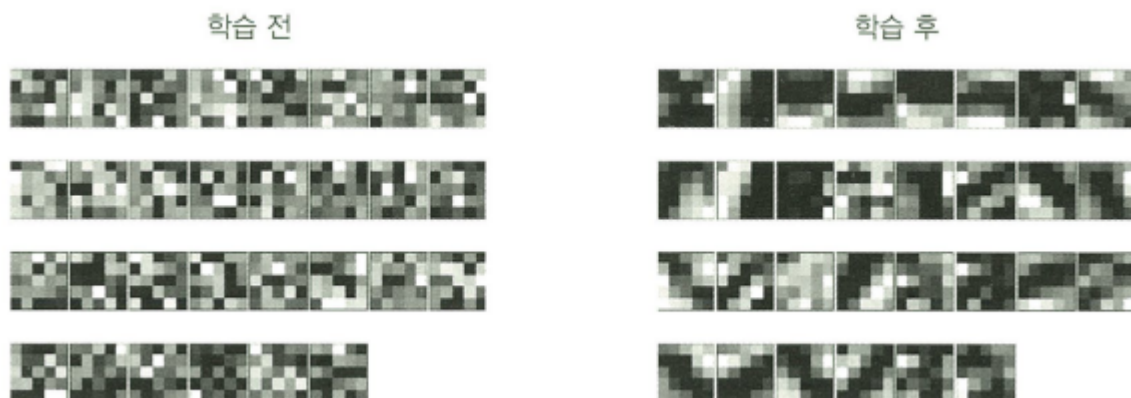
7.6. CNN 시각화하기

CNN은 어떤 것을 보고 있는지 그림으로 알아보자

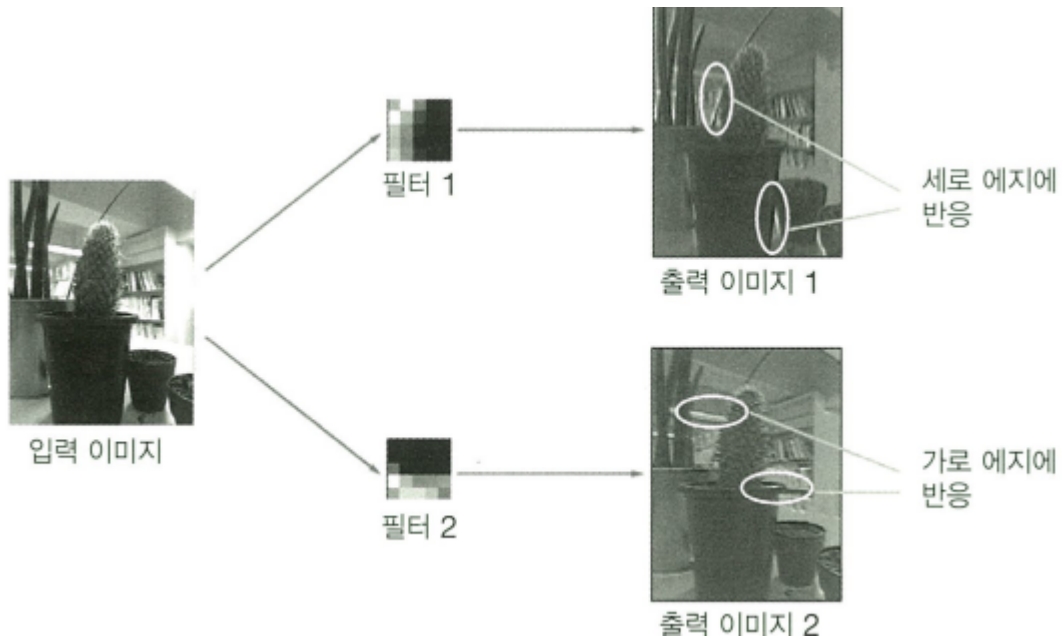
7.6.1. 1번째 층의 가중치 시각화하기

1번째 층의 합성곱 계층의 가중치가 (30, 1, 5, 5)이므로 흑/백 표현 가능

합성곱 계층 필터를 이미지로 나타내면?



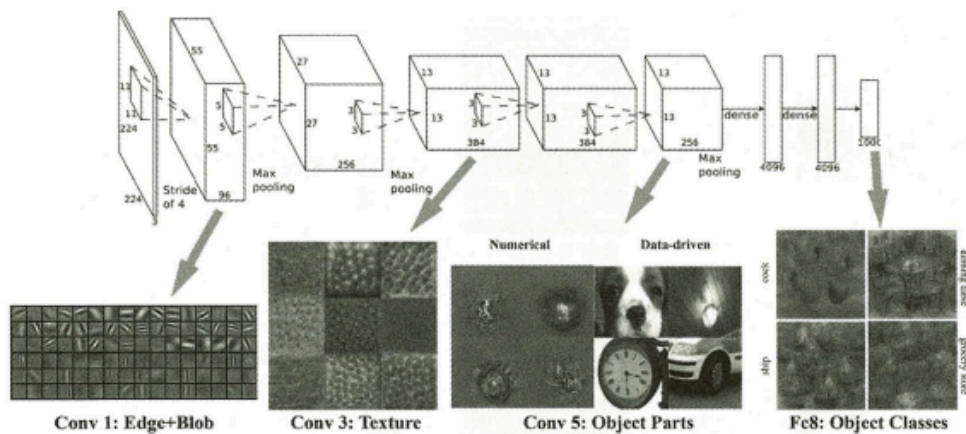
우측의 필터가 보고 있는 건 에지(색상 변경의 경계선), 블롭(국소적으로 덩어리진 영역) 등
등



7.6.2. 층 깊이에 따른 추출 정보 변화

깊어지면 깊어질수록 추출되는 정보가 점점 추상화된다.

그림 7-26 CNN의 합성곱 계층에서 추출되는 정보. 1번째 층은 에지와 블롭, 3번째 층은 텍스처, 5번째 층은 사물의 일부, 마지막 완전연결 계층은 사물의 클래스(개, 자동차 등)에 뉴런이 반응한다.^[19]



합성곱 계층을 여러 겹 쌓게 되면, 층이 깊어지면서 더 복잡하고 추상화된 정보가 추출된다. 처음 층은 단순한 에지에 반응하고, 텍스처에 반응하고, 더 복잡한 사물의 일부에 반응한다. 이 말은 즉슨, 뉴런이 반응하는 대상이 단순한 모양 반응에서 "고급"정보로 변화해간다는 것
⇒ 사물의 "의미"를 이해하도록 변화함

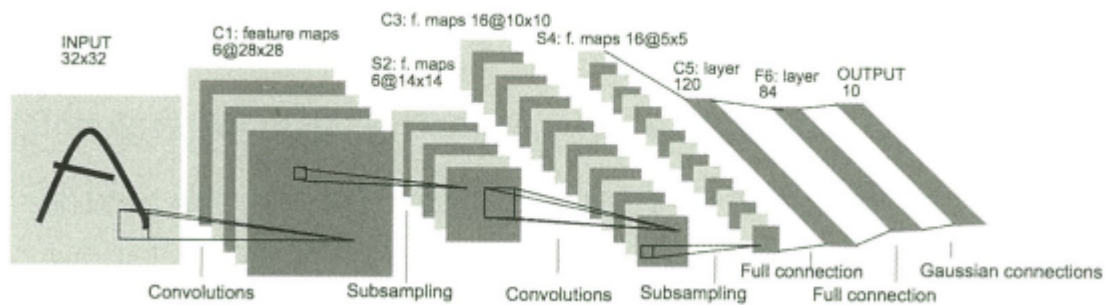
7.7. 대표적인 CNN

1. CNN의 원조인 LeNet
2. 딥러닝의 주목을 이끈 AlexNet

7.7.1. LeNet

단순히 서브 샘플링 계층 원소 줄이기만을 반복하고 마지막으로 완전연결 계층을 거치며 결과를 출력

그림 7-27 LeNet의 구성^[20]

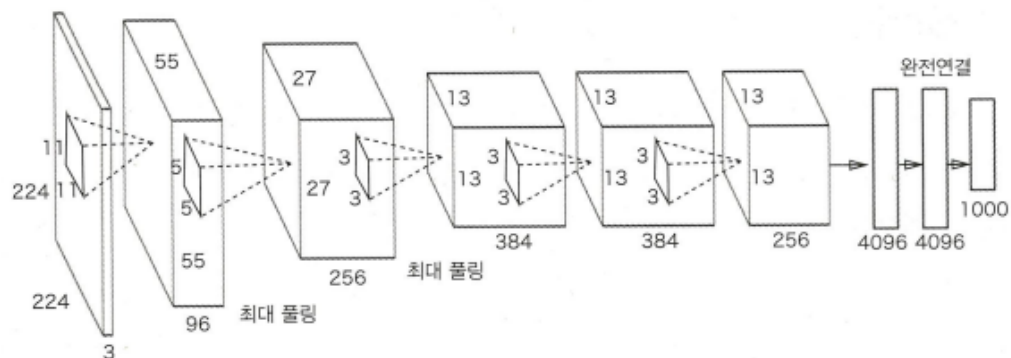


손글씨 숫자를 인식하는 네트워크로 제안된 네트워크

7.7.2 AlexNet

기본적인 구성은 유사하다

그림 7-28 AlexNet의 구성^[21]



변화된 것:

1. 활성화 함수로 ReLU 이용
2. LRN이라는 국소적 정규화를 실시하는 계층 이용
3. 드롭 아웃 사용

7.8. 정리

1. CNN :

완전연결 계층 네트워크 + **합성곱 계층** + **풀링 계층**

2. 합성곱 계층과 풀링 계층은 이미지를 행렬로 전개하면 더 간단하고 효율적으로 구현할 수 있다

→ im2col이 이미지를 행렬로 전개하는 것

3. CNN을 시각해봤을때

계층이 깊어질수록 고급 정보 추출

4. 대표적인 CNN

a. LeNet

b. AlexNet

5. 딥러닝의 발전에는 빅데이터와 GPU가 큰 영향을 미쳤다.